

DATABASE MANAGEMENT SYSTEM (DBMS)

Five-Page Academic Study Notes

1. Introduction to DBMS

A Database Management System (DBMS) is software used to create, store, organize, retrieve, update, and manage data efficiently. Instead of keeping information in separate files, a DBMS provides a structured environment in which related data can be stored and accessed by multiple users and applications.

Examples: MySQL, PostgreSQL, Oracle Database, Microsoft SQL Server, and SQLite.

2. Why DBMS is Needed

Traditional file systems can suffer from data redundancy, inconsistency, difficult data sharing, weak security, and poor backup mechanisms. A DBMS addresses these problems by providing controlled access to data and maintaining rules that preserve data quality.

3. Major Advantages

- **Reduced redundancy:** avoids unnecessary duplication of information.
- **Data consistency:** keeps related copies of data synchronized.
- **Security:** provides authentication and authorization mechanisms.
- **Data sharing:** allows multiple applications and users to access data.
- **Backup and recovery:** helps restore data after failures.
- **Concurrency control:** manages simultaneous access by multiple users.

4. DBMS Architecture

A DBMS can be viewed as a layer between users/applications and stored data. Users issue queries or perform operations through applications. The DBMS processes these requests and communicates with the storage system.

Three-level architecture: External level → Conceptual level → Internal level.

The external level represents user views, the conceptual level describes the overall logical structure of the database, and the internal level describes how data is physically stored.

RELATIONAL MODEL AND DATABASE DESIGN

1. Relational Model

The relational model represents data using tables. A table consists of rows and columns. A row represents a record or tuple, while a column represents an attribute of the entity being stored.

Example: STUDENT Table

Student_ID	Name	Department	Semester
101	Aman	CSE	4
102	Riya	IT	4
103	Karan	ECE	3

2. Keys in DBMS

- **Primary Key:** uniquely identifies each record in a relation.
- **Candidate Key:** a minimal attribute set capable of uniquely identifying records.
- **Foreign Key:** an attribute that references a key in another table.
- **Composite Key:** a key formed using two or more attributes.

3. Integrity Constraints

Integrity constraints are rules that maintain correctness and validity of database data. Common constraints include NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, and CHECK constraints.

4. Entity-Relationship (ER) Model

The ER model is used during database design. An entity represents a real-world object, an attribute describes a property of that object, and a relationship represents an association between entities. ER diagrams help designers understand the structure before implementing tables.

5. Relationships

- **One-to-One:** one entity is associated with one entity.
- **One-to-Many:** one entity can be associated with many entities.
- **Many-to-Many:** many entities can be associated with many entities.

SQL AND DATABASE OPERATIONS

1. Structured Query Language (SQL)

SQL is a standard language used to interact with relational databases. It can be used to create database objects, insert and modify records, retrieve information, and control access.

2. Categories of SQL Commands

DDL – Data Definition Language: CREATE, ALTER, DROP, TRUNCATE.

DML – Data Manipulation Language: INSERT, UPDATE, DELETE.

DQL – Data Query Language: SELECT.

DCL – Data Control Language: GRANT, REVOKE.

TCL – Transaction Control Language: COMMIT, ROLLBACK, SAVEPOINT.

3. Common SQL Examples

Create a table:

```
CREATE TABLE Student (Student_ID INT PRIMARY KEY, Name VARCHAR(50), Department VARCHAR(20));
```

Insert data:

```
INSERT INTO Student VALUES (101, 'Aman', 'CSE');
```

Retrieve data:

```
SELECT * FROM Student WHERE Department = 'CSE';
```

Update data:

```
UPDATE Student SET Department = 'IT' WHERE Student_ID = 101;
```

Delete data:

```
DELETE FROM Student WHERE Student_ID = 101;
```

4. Joins

Joins combine rows from multiple tables based on a related column. Common types include INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN.

For example, a Student table can be joined with a Course table using Student_ID. This allows a query to retrieve student information together with related course information.

5. Views

A view is a virtual table based on a SQL query. Views can simplify complex queries, provide customized representations of data, and help restrict access to selected columns or rows.

NORMALIZATION AND TRANSACTIONS

1. Database Normalization

Normalization is a database design technique used to organize data and reduce undesirable redundancy and dependency. It divides large tables into smaller related tables while preserving relationships.

2. Normal Forms

First Normal Form (1NF): Each attribute contains atomic values and repeating groups are removed.

Second Normal Form (2NF): The relation is in 1NF and every non-key attribute is fully dependent on the whole primary key.

Third Normal Form (3NF): The relation is in 2NF and non-key attributes do not depend transitively on the primary key.

BCNF: A stronger form of 3NF in which every determinant is a candidate key.

3. Functional Dependency

A functional dependency describes a relationship between attributes. If attribute X determines attribute Y, it is represented as $X \rightarrow Y$. Functional dependencies are important in identifying keys and performing normalization.

4. Transactions

A transaction is a logical unit of database work. For example, transferring money from one bank account to another involves debiting one account and crediting another. Both operations should be treated as one consistent unit.

5. ACID Properties

- **Atomicity:** a transaction is completed fully or not performed.
- **Consistency:** a transaction preserves database rules and constraints.
- **Isolation:** concurrent transactions should not improperly interfere with each other.
- **Durability:** committed changes survive system failures.

6. Concurrency Control

Concurrency control ensures correct results when multiple transactions execute simultaneously. Techniques include locking, timestamp ordering, and multiversion concurrency control.

A lock may be shared for reading or exclusive for writing. Proper locking helps prevent problems such as lost updates, dirty reads, and inconsistent results.

INDEXING, RECOVERY AND REVISION

1. Indexing

An index is an auxiliary data structure that helps the DBMS locate records efficiently. Without an appropriate index, the database may need to scan many rows to find matching data.

Common Index Structures

- **B-Tree / B+ Tree:** widely used for ordered access and range queries.
- **Hash Index:** useful for fast equality-based lookups.
- **Primary Index:** associated with a primary ordering key.
- **Secondary Index:** created on a non-primary attribute to speed up searches.

2. Database Recovery

Recovery techniques protect the database against failures such as power loss, software errors, or hardware failures. The goal is to restore the database to a correct and consistent state.

Log-based recovery records changes made by transactions. During recovery, the system can use the log to redo committed operations and undo incomplete operations.

3. Database Security

Database security protects information from unauthorized access and accidental or malicious modification. Important mechanisms include authentication, authorization, roles, privileges, encryption, auditing, and secure backup practices.

4. Quick Revision Points

- DBMS manages structured data and provides controlled access.
- Relational databases store information in tables.
- Primary keys uniquely identify records.
- Foreign keys maintain relationships between tables.
- SQL is used to define, manipulate, query, and control database data.
- Normalization reduces redundancy and update anomalies.
- ACID properties describe reliable transaction behavior.
- Indexes improve data retrieval performance.
- Recovery mechanisms help restore databases after failures.

5. Conclusion

DBMS is a fundamental component of modern software systems because it provides systematic storage, retrieval, security, consistency, concurrency control, and recovery of data. Understanding relational models, SQL, normalization, transactions, indexing, and recovery provides a strong foundation for advanced database development and backend engineering.

End of 5-Page DBMS Notes